ETHICALHACKER / USER GUIDE

# Create, share and play content packs

A practical guide to the PC content editor and to loading community packs on PC and Web.

**For** scenario authors and players using an exported EthicalHacker build.

**In this guide** local editing, five stage types, pack structure, sharing through a public GitHub repository, and loading the shared pack in the Web version.

**At a glance** Create and edit packs in the PC build. Players can load a public online pack by URL on PC or Web. A Web player cannot save or edit a pack in the browser.

Guide edition: September 2026. Steps reflect the current EthicalHacker source. The exported PC and Web builds were not run to validate these instructions.

# 1. Choose the right workflow

| Your goal | Use this route |
| --- | --- |
| Edit your own scenario | PC build: Main Menu > Editor > local pack > scenario. |
| Install a downloaded pack | PC build: place its folder in StreamingAssets, then reopen the pack list. |
| Play a shared pack | PC or Web: enter its public GitHub pack URL on the New Game screen. |
| Share your pack | Publish manifest.json and scenario JSON files in one public GitHub repository folder; share that folder URL. |

**Important distinction** The editor lists local folders. The New Game screen can also load an online pack. Entering a GitHub URL lets players run a scenario; it does not import the remote pack into the editor.

## What a content pack contains

A pack is a folder with a **manifest.json** file. The manifest identifies the pack and lists scenario JSON files. Each scenario holds its ordered stages, optional skills and optional points of interest (POIs). A ZIP archive alone is not a pack URL for the game.

| Layer | Example | Role |
| --- | --- | --- |
| Pack folder | MyPack/ | One folder under StreamingAssets, or one public repository folder. |
| Manifest | MyPack/manifest.json | Pack ID, name, version and scenario file list. |
| Scenario | MyPack/intro.json | Title, description, stages, skills and POIs. |

## PC versus Web

The PC build can enumerate local folders in StreamingAssets and save local scenarios. The Web build uses online requests for packs and has no local filesystem editing support. Both versions expose the online loader from the New Game screen in the current code.

# 2. Open the PC editor

- Launch the standalone PC game and choose **Editor** from the main menu.
- On the content selection screen, select the folder name of a local pack. The editor reads its manifest and the scenario files listed there.
- Select a scenario. Its fields appear in the inspector. Edit its name, description, stages, skills and POIs as needed.
- Choose **Back** from the scenario screen to save the local pack. Saving writes the scenario JSON files and rewrites the manifest's scenario list.

> **Create New Content Pack** The current button is unfinished. To start a pack, create its folder and manifest.json manually as shown in section 3, then return to the content selection screen or reopen the editor.

## Find the local content folder

Use the editor's **Open Content Folder** control when available. In a typical Windows Unity build the same location is beside the executable under **<GameName>_Data/StreamingAssets**. Open the folder from the game when in doubt; the exact executable name can differ between builds.

## Saving and checking your work

- Finish editing one scenario, return with **Back**, and check that its JSON file and manifest exist in the pack folder.
- Keep a separate copy before editing a pack you received from someone else. Saving can rewrite its manifest and listed scenario files.
- Start a new game from the edited local pack to review text, choices, answers, map markers and progression. Do not assume a valid JSON file proves the scenario plays correctly.

# 3. Build a new local pack

Create a uniquely named folder in StreamingAssets. Put the manifest and scenario files directly inside that folder. The manifest's **file** entries must name files that the game can fetch from that same folder.

```
StreamingAssets/
  MyPack/
    manifest.json
    intro.json
    next_chapter.json
```

## Example manifest.json

```
{
  "contentId": "MyPack",
  "contentName": "My Training Pack",
  "version": "1.0.0",
  "scenarios": [
    { "name": "Getting Started", "file": "intro.json" },
    { "name": "Next Chapter", "file": "next_chapter.json" }
  ]
}
```

Use the same pack ID in the folder name, **contentId** and each scenario's **contentPackId**. This keeps editor folder actions and saved scenario references consistent. The scenario file names in the manifest must match the actual files, including their extension and letter case on case-sensitive hosts.

## Add the first scenario

- Create a small JSON scenario file first, or select the pack in the PC editor and use **Create New Scenario**. Give it a unique scenario ID.
- Set a player-facing name and description. Add at least one stage using the editor's stage buttons.
- After editing, use **Back** so the editor saves the scenario and updates manifest.json. Open the written files to confirm the expected ID and file name.

> **Existing examples** The shipped BaseContent and ExtraContent folders demonstrate manifest and scenario structure. Copy only your own material or content you have permission to redistribute.

# 4. Work with scenarios and stages

A scenario has an **id**, **contentPackId**, player-facing **name** and **description**, plus lists of **stages**, **skills** and **pointsOfInterest**. In the editor, scenario and pack IDs are locked after creation; plan stable IDs before writing content.

## Stage controls

The stage area provides **Add Dialogue**, **Add Decision**, **Add Question**, **Add Skill Unlock** and **Add Map Interaction**. Each new stage receives an ID. Use the visible move up, move down and delete controls to set the play order. The list order is the scenario's stage order.

| Type | Use it for | Fields to check |
|---|---|---|
| DIALOGUE | Story, instructions and feedback. | id, characterName, stageText, characterSpriteId. |
| DECISION | A player choice with consequences. | id, characterName, stageText; options with optionText, moneyReward and honorReward. |
| QUESTION | A knowledge check. | id, characterName, stageText; answers, zero-based correctAnswerIndex, incorrectAnswerText, moneyReward and legendReward. |
| UNLOCK_SKILL | Introduce a skill used later. | id, characterName, stageText and skillId matching an entry in scenario.skills. |
| MAP_INTERACTION | A story location on the globe. | id, characterName, stageText, latitude, longitude, locationId and iconId. |

**Stage type is fixed** Choose the correct Add button for each new stage. The editor creates the matching stage data type; stageType is not a free-text instruction to the game.

# 5. Author the five stage types

## Dialogue

Use a short **stageText** to establish a situation or explain a result. Set **characterName** for the speaker and a valid **characterSpriteId** when the dialogue should show a portrait. Example: an analyst introduces the organization and the authorized task.

## Decision

Add meaningful options to the **options** list. Each option has its own **optionText**, **moneyReward** and **honorReward**. Test each branch in the game, especially when a reward or consequence should differ. Example: disclose a finding responsibly or postpone the report.

## Question

Enter the possible **answers**, then set **correctAnswerIndex** to the position of the correct answer starting at **0**. If the second answer is correct, use index **1**. Supply **incorrectAnswerText** so the player receives useful feedback; set the configured money and legend rewards.

## Unlock Skill

Add the skill to the scenario's **skills** list with an ID, name, description and cost. Set the stage's **skillId** to exactly that ID. The editor highlights an unlock stage whose skill reference does not match a skill entry.

## Map Interaction

Use a story-relevant latitude and longitude and provide **locationId** and **iconId** values supported by the game. Test the resulting globe marker in play. An optional side event belongs in **pointsOfInterest** instead of being duplicated as a story stage.

# 6. Skills and points of interest

## Skills

Use **Add Skill** in the scenario inspector, then fill in its **id**, **name**, **description** and **cost**. Keep IDs unique within the scenario. If a stage unlocks the skill, its **skillId** must refer to this entry exactly.

## Points of interest

A POI is a separate optional map event. Give it an ID, title, description, latitude, longitude and **showAtProgressPercent**. It can award money, honor and legend. Its optional decisions contain option text and their own rewards.

| Story marker | Optional POI |
| --- | --- |
| A MAP_INTERACTION stage in the ordered stages list. | An entry in pointsOfInterest, outside the main stage sequence. |
| Use for a required story location. | Use for a side event that reinforces the scenario. |
| Needs locationId and iconId. | Needs title, description and progress threshold. |

**Review in play** Check that coordinates, marker visibility, progress timing, choice rewards and skill unlocking behave as intended. Source inspection alone does not verify game presentation.

# 7. Install a pack on a PC

- Download the pack from a source you trust and extract it. A ZIP archive must be unpacked first.
- Locate the folder that directly contains **manifest.json**. If the download added an extra repository-name directory, move the actual pack folder rather than leaving a nested path.
- Copy that pack folder into the game's **StreamingAssets** directory. Keep each pack in a separate folder; do not merge scenario files into BaseContent.
- Launch or return to the game. In the editor, revisit the pack selection screen; in New Game on PC, use **Reload Content** if the list was already open.
- Select the pack and a scenario. If it does not appear, check the folder structure, manifest.json and exact scenario file names.

**Before replacing files** Copy the original pack folder to a safe location. A replacement may change scenario IDs or stage order used by an existing save.

## Local pack check

| Check | Expected result |
| --- | --- |
| Folder | StreamingAssets/MyPack/manifest.json exists. |
| Manifest | scenarios lists every scenario file to load. |
| IDs | contentId and contentPackId identify the intended pack. |
| JSON | Manifest and scenario files parse as JSON. |
| Play | The scenario opens and its stages advance in the expected order. |

# 8. Publish your pack on GitHub

Any author can share their own pack through a **public** GitHub repository. The game fetches JSON files by URL; it does not download a GitHub ZIP or clone the repository. Publish only material you are allowed to share, and keep the repository public so other players can read its files without signing in.

- Create a public repository on GitHub. Upload **manifest.json** and every scenario file listed in it, either at the repository root or together in one subfolder.
- Commit the files. Open the manifest and one scenario in the browser to confirm they are visible publicly and that names and letter case match.
- Share the URL of the **folder containing manifest.json**. For a root-level pack on a **main** branch, the repository URL is accepted by the current loader. For a subfolder or another branch, use the GitHub **tree** URL that names the branch and folder.

## Accepted URL patterns

```
https://github.com/OWNER/REPO
https://github.com/OWNER/REPO/tree/BRANCH/PACK_FOLDER
https://raw.githubusercontent.com/OWNER/REPO/BRANCH/PACK_FOLDER
```

The root repository form is converted by the game to the **main** branch. If the repository uses another branch, share an explicit **tree/BRANCH** URL. The URL must point to the pack folder, not to manifest.json, a **blob** page, a ZIP download or an issue attachment.

> **Stable links matter** An online save records the source URL and scenario ID, then loads the pack again when resumed. Moving, renaming, making private or deleting a published pack can break that save.

# 9. Load a shared pack in Web or PC

## Web player steps

- Open the hosted EthicalHacker Web game. Choose **Play** and an empty save slot to reach **New Game**.
- Paste the public URL of the pack folder into the GitHub URL field and choose **Load Content from GitHub**.
- Wait for the pack header and its scenarios to appear. Choose a scenario to start playing.
- On a later visit, select the save slot. The game reloads the pack from its recorded URL, so the public files must remain reachable.

## PC player steps

The same URL field on **New Game** loads a public pack on PC. A PC player may instead install a local copy under StreamingAssets. To edit an online pack, download it as a local pack first, then open that local folder in the PC editor.

## When Web loading fails

Check that the URL points to the folder with **manifest.json**, the repository is public, and every listed scenario file is reachable. A browser also applies cross-origin access rules to requests from a Web game: the remote host must permit the request. Check the browser console or ask the game host for help if the URLs open directly but the game cannot fetch them.

> **Web limitation** The Web version loads public online content for play. It does not browse local pack folders or save edits to a GitHub repository.

# 10. Troubleshooting and release checklist

| Symptom | What to inspect |
| --- | --- |
| Pack does not appear on PC | Check its folder is directly under StreamingAssets; reopen the list or use Reload Content in New Game. |
| Online pack does not appear | Use a public pack-folder URL; check manifest.json and browser access to every listed file. |
| Some scenarios are missing | Check each manifest file value and the corresponding file name; scenario load errors can leave the rest visible. |
| Edits did not persist | Use Back from the scenario screen to save; inspect the local JSON and manifest afterward. Online packs are read-only in the editor. |
| Saved game no longer loads | Check that the source URL, public access, pack files and original scenario ID are still available. |
| Skill or answer acts incorrectly | Check skillId against skills.id; check zero-based correctAnswerIndex and test in play. |

## Before sharing

- Manifest and every listed scenario are valid JSON and publicly accessible.
- Pack ID, scenario IDs and file names are stable and consistent.
- Stages are in play order; every question and decision has been reviewed.
- Skill links, POIs and map positions have been checked in the game where a build is available.
- The shared URL names the pack folder; the branch will remain available.

## Reference links

GitHub: Create a repository | GitHub: Add files | Unity: Web networking and CORS

This guide describes the current source behavior. It does not certify the state of any particular exported build or public content repository.